Clinvio Nucleus Framework — AI Agent Skill Guide

Overview

Clinvio Nucleus is a 34-module enterprise Java framework for building secure, server-rendered web applications. It uses Spring Boot 3.3.5, Java 21, Thymeleaf templates, HTMX for partial page updates, and SQLite/PostgreSQL with AES-256-GCM field-level encryption.

Maven Coordinates: `hu.clinvio:clinvio-nucleus:2.0.0-SNAPSHOT` **Repository:** `https://dev.clinvio.hu/api/packages/jokerz/maven`

Quick Reference

Minimum Dependencies

```
<repositories>
    <repository>
        <id>gitea</id>
        <url>https://dev.clinvio.hu/api/packages/jokerz/maven</url>
    </repository>
</repositories>

<dependencies>
    <dependency>
        <groupId>hu.clinvio</groupId>
        <artifactId>clinvio-nucleus-spring-boot-starter</artifactId>
        <version>2.0.0-SNAPSHOT</version>
    </dependency>
    <dependency>
        <groupId>hu.clinvio</groupId>
        <artifactId>clinvio-nucleus-persistence</artifactId>
        <version>2.0.0-SNAPSHOT</version>
    </dependency>
    <dependency>
        <groupId>hu.clinvio</groupId>
        <artifactId>clinvio-nucleus-business</artifactId>
        <version>2.0.0-SNAPSHOT</version>
```

```
    </dependency>
</dependencies>
```

Minimum Configuration (SQLite)

```
spring.datasource.url=jdbc:sqlite:./data/app.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.jpa.hibernate.ddl-auto=update
clinvio.persistence.encryption-key=your-secret-key
```

PostgreSQL Configuration

```
spring.datasource.url=jdbc:postgresql://localhost:5432/clinvi
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=update
clinvio.nucleus.postgresql.enabled=true
clinvio.persistence.encryption-key=${CLINVIO_ENCRYPTION_KEY}
```

Package Structure

- `hu.clinvio.ui.*` — Core framework classes (persistence, htmx, components, business)
- `hu.clinvio.nucleus.*` — Domain module classes (appointment, tasks, ai, etc.)

Module Reference (34 modules)

Core Modules (Required)

Module	Purpose
`clinvio-nucleus-spring-boot-starter`	Auto-configuration for all modules
`clinvio-nucleus-persistence`	BaseEntity, @SensitiveData encryption, SQLite/PostgreSQL
`clinvio-nucleus-business`	Services, controllers, workflow, validation, exceptions

Infrastructure Modules

Module	Purpose

`clinvio-nucleus-security`	JWT authentication, @Secured, BCrypt
`clinvio-nucleus-observability`	Micrometer metrics, health checks, request logging
`clinvio-nucleus-cache`	Caffeine in-memory caching with TTL
`clinvio-nucleus-migration`	Flyway database migrations
`clinvio-nucleus-openapi`	Swagger UI auto-generation
`clinvio-nucleus-scheduler`	Cron task scheduling
`clinvio-nucleus-mail`	SMTP email with templates
`clinvio-nucleus-websocket`	WebSocket support
`clinvio-nucleus-devtools`	Hot reload, debug endpoints, code scaffolding
`clinvio-nucleus-ratelimit`	Token bucket rate limiting for API endpoints
`clinvio-nucleus-pdf`	PDF generation utilities
`clinvio-nucleus-docker`	Dockerfile and docker-compose generation
`clinvio-nucleus-postgresql`	PostgreSQL database support
`clinvio-nucleus-redis`	Redis cache integration
`clinvio-nucleus-s3`	S3-compatible object storage
`clinvio-nucleus-search`	Elasticsearch integration

Domain Modules

Module	Purpose
`clinvio-nucleus-compliance`	GDPR consent management
`clinvio-nucleus-appointment`	Appointment scheduling
`clinvio-nucleus-documents`	File storage and management
`clinvio-nucleus-tasks`	Task/project management
`clinvio-nucleus-ai`	AI integration (OpenAI)
`clinvio-nucleus-multitenancy`	Tenant isolation
`clinvio-nucleus-notifications`	Multi-channel notifications (email, SMS, push)
`clinvio-nucleus-test`	Test utilities (TestDataFactory, CvBaseServiceTest)

Tools

Module	Purpose
`clinvio-cli`	CLI tool for project scaffolding
`clinvio-maven-plugin`	Maven plugin for code generation

Entity Pattern

Basic Entity

```
import hu.clinvio.ui.persistence.entity.BaseEntity;
import jakarta.persistence.*;

@Entity
@Table(name = "orders")
public class Order extends BaseEntity {
    @Column(nullable = false, unique = true)
    private String orderNumber;

    @Column(nullable = false)
    private String customerName;

    @Enumerated(EnumType.STRING)
    private OrderStatus status;

    private BigDecimal totalAmount;

    // Getters and setters...
}
```

BaseEntity provides: `id` (Long), `createdAt`, `updatedAt`, `deletedAt` (soft delete).

Encrypted Entity

```
import hu.clinvio.ui.persistence.annotation.SensitiveData;
import hu.clinvio.ui.persistence.crypto.EncryptedStringConverter;

@Entity
public class Customer extends BaseEntity {
    @Column(nullable = false, unique = true)
    private String customerNumber;  // NOT encrypted, searchable

    @SensitiveData
    @Convert(converter = EncryptedStringConverter.class)
    @Column(nullable = false, length = 500)
    private String name;  // Encrypted with AES-256-GCM

    @SensitiveData
```

```
    @Convert(converter = EncryptedStringConverter.class)
    private String email;  // Encrypted

    @SensitiveData
    @Convert(converter = EncryptedStringConverter.class)
    private String phone;  // Encrypted

    // Getters and setters...
}
```

IMPORTANT: Always use both `@SensitiveData` AND `@Convert` on encrypted fields. JPA does not process meta-annotations.

IMPORTANT: Encrypted fields cannot be searched with SQL LIKE. Always have a non-encrypted business identifier for searching.

Repository Pattern

```
import hu.clinvio.ui.business.repository.CvBaseRepository;

@Repository
public interface OrderRepository extends CvBaseRepository<Order, Long> {
    @Query("SELECT o FROM Order o WHERE o.deletedAt IS NULL AND o.orderNu
    Page<Order> search(@Param("query") String query, Pageable pageable);
}
```

CvBaseRepository provides: `findAllActive()`, `findAllActive(Pageable)`, `findAllActive(Sort)`, `findActiveById()`, `softDelete()`, `countActive()`, `existsActiveById()`, `findCreatedSince()`, `findRecent()`, `search()`.

Service Pattern

```
import hu.clinvio.ui.business.service.CvSearchableService;

@Service
@Transactional(readOnly = true)
public class OrderService extends CvSearchableService<Order, Long> {

    public OrderService(OrderRepository repo) {
```

```
        super(repo, repo);
    }

    @Override
    public Page<Order> search(String query, Pageable pageable) {
        if (query == null || query.isBlank()) return findAll(pageable);
        return ((OrderRepository) repository).search(query, pageable);
    }

    @Transactional
    public Order updateStatus(Long id, OrderStatus newStatus) {
        Order order = getById(id);
        order.setStatus(newStatus);
        return repository.save(order);
    }

    @Override
    protected String getEntityName() { return "Order"; }
}
```

CvBaseService provides: `findAll()`, `findAll(Pageable)`, `findAll(Sort)`, `findById()`, `getById()`, `create()`, `createAll()`, `update()`, `updateAll()`, `delete()`, `deleteAll()`, `count()`, `existsById()`, `findRecent()`, `findPage()`, `updateField()`, `restore()`, `countCreatedAfter()`.

Validation hooks: `validateBeforeCreate()`, `validateBeforeUpdate()`, `validateBeforeDelete()`, `canDelete()`.

Controller Pattern

```
import hu.clinvio.ui.business.controller.CvBaseCrudController;

@Controller
@RequestMapping("/orders")
public class OrderController extends CvBaseCrudController<Order> {

    public OrderController(OrderService service) { super(service); }

    @Override protected String getListTemplate() { return "orders/list";
    @Override protected String getDetailTemplate() { return "orders/deta
```

```
    @Override protected String getEntityName() { return "Order"; }
    @Override protected String getEntityPath() { return "/orders"; }

    @Override
    protected CvDataTable<Order> buildDataTable(Page<Order> page) {
        return new CvDataTable<Order>()
            .addColumn("orderNumber", "Order #", true)
            .addColumn("customerName", "Customer", true)
            .addColumn("totalAmount", "Total")
            .addColumn("status", "Status", true)
            .data(page.getContent())
            .searchable(true).pageable(true).pageSize(page.getSize());
    }

    @GetMapping
    public String listView(Model model, Pageable pageable) {
        return super.listView(model, pageable);
    }

    @GetMapping("/{id}")
    public String detailView(@PathVariable Long id, Model model) {
        return super.detailView(id, model);
    }
}
```

IMPORTANT: Always add `@GetMapping` to `listView()` and `detailView()` in subclasses.

REST API Controller

```
@RestController
@RequestMapping("/api/orders")
public class OrderApiController extends CvBaseApiController<Order> {
    public OrderApiController(OrderService service) { super(service); }
    @Override protected String getEntityName() { return "Order"; }
    @Override protected String getEntityPath() { return "/api/orders"; }
}
```

Auto-generates: GET (list), GET/{id}, POST, PUT/{id}, DELETE/{id}, /all, /count, /{id}/exists.

HTMX Responses

```
// Toast + Redirect
return HxResponse.builder()
    .trigger("cv:toast", Map.of("message", "Created", "type", "success"))
    .redirect("/orders/" + id)
    .build();

// Fragment swap
return HxResponse.builder()
    .swap(HxSwap.OUTER_HTML)
    .fragment("orders/fragments :: status-badge")
    .model(Map.of("status", newStatus))
    .build();

// Redirect only
return HxResponse.builder().redirect("/orders").build();
```

Workflow Engine

```
WorkflowEngine<OrderStatus> workflow = WorkflowEngine.of(OrderStatus.cla
    .transition(PENDING, PROCESSING, CANCELLED)
    .transition(PROCESSING, SHIPPED, CANCELLED)
    .transition(SHIPPED, DELIVERED)
    .onTransition(PROCESSING, SHIPPED, (from, to) -> {
        notificationService.send(userId, "Shipped", "Your order is on th
    })
    .build();

workflow.validate(currentStatus, newStatus);  // throws if invalid
workflow.transition(order, newStatus, order::setStatus);  // with callba
```

Validation

```
CvValidationResult result = CvValidation.builder()
    .require(name, "name", "Name is required")
    .requireEmail(email, "email")
```

```
        .requireMinLength(password, 8, "password")
        .requirePositive(amount, "amount")
        .build();

    if (result.hasErrors()) {
        throw new CvValidationException("Invalid input", result);
    }
```

Error Handling

Exception	HTTP	Use
`CvNotFoundException`	404	Entity not found
`CvDuplicateException`	409	Duplicate entity
`CvBusinessException`	422	Business rule violation
`CvValidationException`	400	Field validation errors
`CvCryptoException`	500	Encryption failure

Rate Limiting

```
@RateLimiter(requests = 100, durationSeconds = 60)
@GetMapping("/api/orders")
public ResponseEntity<List<Order>> list() { ... }
```

Configuration:

```
clinvio.nucleus.ratelimit.enabled=true
clinvio.nucleus.ratelimit.default-requests=100
clinvio.nucleus.ratelimit.default-duration-seconds=60
```

Notifications

```
@Autowired CvNotificationService notificationService;

notificationService.send(userId, "Order Shipped", "Your order is on the v
```

```
notificationService.sendToChannel("email", userId, "Welcome", "Hello!");
notificationService.sendSuccess(userId, "Success", "Operation completed");
```

Appointments

```
@Autowired CvAppointmentService appointmentService;

CvAppointment apt = new CvAppointment();
apt.setTitle("Doctor Visit");
apt.setStartTime(LocalDateTime.now().plusDays(1));
apt.setEndTime(LocalDateTime.now().plusDays(1).plusHours(1));
appointmentService.create(apt);

appointmentService.reschedule(id, newStart, newEnd);
appointmentService.cancel(id);
appointmentService.confirm(id);
```

Documents

```
@Autowired CvDocumentService documentService;

String path = documentService.store(multipartFile, "orders/" + orderId);
InputStream stream = documentService.retrieve(path);
documentService.delete(path);
```

Tasks

```
@Autowired CvTaskService taskService;

CvTask task = new CvTask();
task.setTitle("Implement feature");
task.setPriority(CvTask.TaskPriority.HIGH);
task.setAssigneeId(userId);
taskService.create(task);
```

```
taskService.updateStatus(id, CvTask.TaskStatus.IN_PROGRESS);
List<CvTask> overdue = taskService.findOverdue();
```

AI Integration

```
@Autowired CvAiService aiService;

String response = aiService.generateText("Explain quantum computing");
String summary = aiService.summarize(longText, 100);
String translated = aiService.translate(text, "Hungarian");
```

Compliance (GDPR)

```
@Autowired CvConsentService consentService;

consentService.grant(userId, CvConsent.ConsentType.DATA_PROCESSING, ipAd
boolean hasConsent = consentService.verify(userId, CvConsent.ConsentType
consentService.revoke(userId, CvConsent.ConsentType.MARKETING);
```

PDF Generation

```
@Autowired CvPdfService pdfService;

byte[] pdf = pdfService.generateSimplePdf("Invoice", "Order #1234");
pdfService.savePdf(pdf, Path.of("invoice.pdf"));
```

Docker Support

```
@Autowired CvDockerService dockerService;

dockerService.writeDockerFile("myapp", "app.jar", 8080, outputPath);
dockerService.writeDockerCompose("myapp", 8080, outputPath);
```

S3 Storage

```
@Autowired CvS3StorageService s3Service;

String key = s3Service.upload("orders/123/invoice.pdf", inputStream, "ap
InputStream stream = s3Service.download(key);
s3Service.delete(key);
String url = s3Service.getPresignedUrl(key, 60);  // 60 minute expiry
```

Configuration:

```
clinvio.nucleus.s3.enabled=true
clinvio.nucleus.s3.endpoint=https://s3.amazonaws.com
clinvio.nucleus.s3.access-key=${AWS_ACCESS_KEY}
clinvio.nucleus.s3.secret-key=${AWS_SECRET_KEY}
clinvio.nucleus.s3.bucket=clinvio-documents
clinvio.nucleus.s3.region=us-east-1
```

Elasticsearch

```
@Autowired CvSearchService searchService;

List<Order> results = searchService.search(query, Order.class);
searchService.index(order);
searchService.delete(order);
```

Configuration:

```
clinvio.nucleus.search.enabled=true
spring.elasticsearch.uris=http://localhost:9200
```

Testing

```
// Test data factory
Order order = TestDataFactory.createEntity(Order.class);
List<Order> orders = TestDataFactory.createEntities(Order.class, 10);
```

```
Page<Order> page = TestDataFactory.createPage(orders, 0, 10);
String email = TestDataFactory.randomEmail();

// Base test class
@SpringBootTest
class OrderServiceTest extends CvBaseServiceTest<Order> {
    @Autowired OrderService orderService;
    @Override protected CvBaseService<Order, Long> getService() { return
    @Override protected Class<Order> getEntityClass() { return Order.cla
}
```

DevTools

```
clinvio.nucleus.devtools.enabled=true
clinvio.nucleus.devtools.hot-reload=true
clinvio.nucleus.devtools.sql-logging=false
clinvio.nucleus.devtools.debug-endpoints=true
```

Debug endpoints: `/debug/info`, `/debug/props`, `/debug/beans`

CLI Tool

```
# Create new project
clinvio init my-app --package=com.example.app

# Generate CRUD module
clinvio generate Order --package=com.example.app --fields="orderNumber:s
```

Maven Plugin

```
<plugin>
    <groupId>hu.clinvio</groupId>
    <artifactId>clinvio-maven-plugin</artifactId>
    <version>2.0.0-SNAPSHOT</version>
    <executions>
        <execution>
```

```
            <goals><goal>generate-crud</goal></goals>
            <configuration>
                <entityName>Order</entityName>
                <basePackage>com.example.app</basePackage>
            </configuration>
        </execution>
    </executions>
</plugin>
```

UI Components

Component	Usage
CvButton	`CvButton.primary("Save").icon("check").hxPost("/api/save")`
CvCard	`CvCard.of().title("Details").body(html)`
CvStatCard	`CvStatCard.of("42", "Total").variant(PRIMARY).icon("box")`
CvDataTable	`new CvDataTable<>().addColumn("name","Name",true).data(list).searchable(true)`
CvModal	`CvModal.of("Confirm").content("Are you sure?").confirmText("Yes")`
CvTabs	`CvTabs.of().addTab("tab1","Tab 1","Content")`
CvAlert	`CvAlert.of("Success!", AlertType.SUCCESS)`
CvTimeline	`CvTimeline.of().addItem("Created","Order created","10:00","bi-plus")`
CvAvatar	`CvAvatar.of("John Smith").size("lg")`
CvProgress	`CvProgress.of(75).max(100).variant("success").showLabel(true)`
CvBadge	`CvBadge.success("Active")` — returns HTML string
CvBreadcrumb	`CvBreadcrumb.of().addItem("Home","/").addActive("Orders")`
CvDropdown	`CvDropdown.of("Actions").addItem("Edit","/edit","bi-pencil")`
CvStepper	`CvStepper.of().addStep("Step 1","Description")`

Configuration Properties

```
# Database (SQLite - default)
spring.datasource.url=jdbc:sqlite:./data/app.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.jpa.hibernate.ddl-auto=update
```

```
# Database (PostgreSQL)
# spring.datasource.url=jdbc:postgresql://localhost:5432/clinvi
# spring.datasource.driver-class-name=org.postgresql.Driver
# clinvio.nucleus.postgresql.enabled=true

# Encryption (REQUIRED)
clinvio.persistence.encryption-key=${CLINVIO_ENCRYPTION_KEY}
clinvio.persistence.database-path=./data/app.db

# Observability
clinvio.nucleus.observability.enabled=true
management.endpoints.web.exposure.include=health,metrics,prometheus

# Cache (Caffeine or Redis)
clinvio.nucleus.cache.enabled=true
clinvio.nucleus.cache.caffeine.maximum-size=1000
clinvio.nucleus.cache.caffeine.expire-after-write=10m
# clinvio.nucleus.redis.enabled=true
# clinvio.nucleus.redis.host=localhost
# clinvio.nucleus.redis.port=6379

# Security
clinvio.nucleus.security.enabled=true
clinvio.nucleus.security.jwt.secret=${JWT_SECRET}
clinvio.nucleus.security.jwt.expiration=86400000

# Rate Limiting
clinvio.nucleus.ratelimit.enabled=true
clinvio.nucleus.ratelimit.default-requests=100
clinvio.nucleus.ratelimit.default-duration-seconds=60

# Scheduler
clinvio.nucleus.scheduler.enabled=true

# Email
clinvio.nucleus.mail.enabled=true
clinvio.nucleus.mail.host=smtp.example.com

# Multi-tenancy
clinvio.nucleus.multitenancy.enabled=true
clinvio.nucleus.multitenancy.tenant-header=X-Tenant-ID
```

```
# S3 Storage
# clinvio.nucleus.s3.enabled=true
# clinvio.nucleus.s3.endpoint=https://s3.amazonaws.com
# clinvio.nucleus.s3.access-key=${AWS_ACCESS_KEY}
# clinvio.nucleus.s3.secret-key=${AWS_SECRET_KEY}
# clinvio.nucleus.s3.bucket=clinvio-documents

# Elasticsearch
# clinvio.nucleus.search.enabled=true
# spring.elasticsearch.uris=http://localhost:9200

# DevTools
clinvio.nucleus.devtools.enabled=true
clinvio.nucleus.devtools.hot-reload=true
clinvio.nucleus.devtools.debug-endpoints=true
```

File Structure

```
my-app/
├── pom.xml
├── src/main/java/com/example/app/
│   ├── Application.java
│   ├── model/
│   │   ├── Order.java
│   │   └── OrderRepository.java
│   ├── service/
│   │   └── OrderService.java
│   └── controller/
│       ├── OrderController.java
│       └── OrderApiController.java
├── src/main/resources/
│   ├── application.properties
│   ├── templates/
│   │   ├── layouts/main.html
│   │   └── orders/
│   │       ├── list.html
│   │       ├── detail.html
│   │       ├── form.html
│   │       └── fragments.html
│   ├── db/migration/
```

```
│   │   └── V1__create_tables.sql
│   └── static/vendor/  (CSS/JS)
└── data/  (SQLite database)
```

Build Commands

```
# Build all modules
./mvnw install -N -DskipTests
./mvnw install -pl clinvio-nucleus-core,clinvio-nucleus-components,... -D

# Run application
./mvnw spring-boot:run -pl my-app

# Deploy to Gitea
./deploy.sh
```